



## Unidade 4 — Por que desenvolver seu próprio Framework?



## Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- **Implementar** um sistema de middlewares;
- **Explicar** o conceito de pipeline de processamento;
- **Aplicar** middlewares para logging e parse de JSON;
- **Analisar** como o Express.js implementa middlewares.

**Taxonomia de Bloom:** Implementar, Explicar, Aplicar, Analisar.



## Conceito de Pipeline

```
Requisição
  ↓
Middleware 1 (logger)    → next()
  ↓
Middleware 2 (json parse) → next()
  ↓
Middleware 3 (auth)      → next()
  ↓
Handler (controller)
  ↓
Resposta
```

- Cada middleware recebe `req`, `res` e `next` ;
- `next()` passa para o próximo middleware;
- Se `next()` não for chamado, a pipeline **para**.



## Implementação do Middleware

```
class MiddlewarePipeline {
  constructor() {
    this.middlewares = [];
  }

  use(fn) {
    this.middlewares.push(fn);
  }

  execute(req, res, finalHandler) {
    let index = 0;

    const next = () => {
      if (index < this.middlewares.length) {
        const middleware = this.middlewares[index++];
        middleware(req, res, next);
      } else {
        // Todos os middlewares executaram → chamar handler
        finalHandler(req, res);
      }
    };

    next(); // Iniciar a pipeline
  }
}
```





## Exemplo: Middleware de Logger

```
function logger(req, res, next) {  
  const data = new Date().toISOString();  
  console.log(`${req.method} ${req.url} - ${data}`);  
  next(); // Passar para o próximo  
}
```



## Exemplo: Middleware de JSON Parse

```
function jsonParse(req, res, next) {  
  if (req.headers['content-type'] === 'application/json') {  
    let body = '';  
    req.on('data', chunk => body += chunk);  
    req.on('end', () => {  
      try {  
        req.body = JSON.parse(body);  
      } catch (e) {  
        req.body = {};  
      }  
      next();  
    });  
  } else {  
    next();  
  }  
}
```



## Integrando Router + Middleware

```
function MiniFramework() {
  const router = new Router();
  const pipeline = new MiddlewarePipeline();

  const app = {
    use(fn) { pipeline.use(fn); },
    get(path, handler) { router.add('GET', path, handler); },
    post(path, handler) { router.add('POST', path, handler); },
    listen(port, cb) {
      const server = http.createServer((req, res) => {
        // Adicionar res.json() e res.status()
        res.json = (data) => {
          res.setHeader('Content-Type', 'application/json');
          res.end(JSON.stringify(data));
        };
        res.status = (code) => { res.statusCode = code; return res; };

        // Executar pipeline → depois router
        pipeline.execute(req, res, () => {
          const match = router.match(req.method, req.url);
          if (match) {
            req.params = match.params;
            match.handler(req, res);
          } else {
            res.statusCode = 404;
            res.json({ erro: 'Rota não encontrada' });
          }
        });
      });
      server.listen(port, cb);
    }
  };

  return app;
}
```





## Síntese da Aula

### Conceitos principais:

1. Middlewares formam uma **pipeline** de processamento;
2. Cada middleware recebe `req`, `res` e `next` ;
3. `next()` passa para o próximo middleware;
4. A integração Router + Middleware é **simples** e **elegante**;
5. O Express usa o **mesmo princípio** internamente.

### Pergunta reflexiva:

Se nosso mini framework já tem Router e Middleware, o que mais o Express tem que nós não temos?

**Próxima aula:** Do Mini Framework ao Framework Real — Vamos comparar e refletir.



